

GENAI Pair Programming Python — Assignment Submission

Task 1: Python Exercises for Senior Engineers (Built-in Libraries Only)

Student Name: Arabindaksha Mishra

Course: Gen AI_PP_Python_03

Language: Python 3.12+ (Pure Standard Library)

Test Status: 100% Passed (40/40 Tests)

Module 1: Unique Elements & Set Algebra

Implements order-preserved list deduplication and set algebra (union, symmetric difference) using pure Python standard library.

```
def get_unique_elements(items: list) -> list:
    seen = set()
    unique = []
    for x in items:
        if x not in seen:
            seen.add(x)
            unique.append(x)
    return unique
```

Module 2: Perfect Number Analyzer

Determines perfect numbers using square-root optimized divisor summation $O(\sqrt{N})$ time complexity.

```
def is_perfect_number(n: int) -> bool:
    if n <= 1: return False
    div_sum = 1
    for i in range(2, int(math.isqrt(n)) + 1):
        if n % i == 0:
            div_sum += i + (n // i if n // i != i else 0)
    return div_sum == n
```

Module 3: Digit Difference Analysis

Calculates absolute difference between max and min digits of integer inputs.

```
def digit_difference(n: int | str) -> int:
    digits = [int(ch) for ch in str(abs(int(n))) if ch.isdigit()]
    return max(digits) - min(digits) if digits else 0
```

Module 4: Fibonacci Series Generator

Provides iterative, memoized recursive (lru_cache), and lazy generator implementations.

```
def fibonacci_iterative(n: int) -> list[int]:
    if n <= 0: return []
    seq = [0, 1]
    while len(seq) < n: seq.append(seq[-1] + seq[-2])
    return seq[:n]
```

Module 5: Anagram Solver

Verifies anagrams using character frequency counting (collections.Counter) and canonical sorting.

```
def is_anagram(s1: str, s2: str) -> bool:
    return Counter(ch.lower() for ch in s1 if ch.isalnum()) == Counter(ch.lower() for ch in s2 if ch.isalnum())
```

Module 6: Movie Ticket Pricing Engine

Age-tiered piecewise ticket pricing algorithm (\$0 infant, \$10 child, \$15 adult, \$12 senior).

```
def calculate_ticket_price(age: int) -> int:
    if age < 3: return 0
    elif age <= 12: return 10
    elif age < 65: return 15
    else: return 12
```

Module 7: Interactive Loops & Sentinels

Sentinel-controlled REPL loop processing and modulo continue skip filtering.

```
def process_topping(topping: str) -> str | None:
    cleaned = topping.strip()
    if not cleaned or cleaned.lower() == 'quit': return None
    return f'Adding {cleaned} to your pizza!'
```